

gnuradio_decode_file.py DSP Mathematics and Implementation Reference

August 12, 2026

Abstract

This document explains the digital signal processing calculations implemented by RetroSpectrum's GNU Radio file-demodulation helper. For every major calculation, it shows the relevant Python source, expresses the same operation mathematically, defines the variables, and explains why the calculation is performed. The emphasis is the active decode path: complex16 IQ normalization, fixed samples-per-symbol averaging, power/envelope extraction, OOK/ASK threshold slicing, robust RAW pulse decoding, rectangular moving-average filtering, phase-discriminator demodulation, SSTV VIS FM-audio preparation and tone scoring, 2-FSK/GFSK/AFSK slicing, 1-D k-means clustering for 4-FSK, ideal PSK/QAM constellation construction, Euclidean nearest-constellation decisions, and final symbol-to-bit conversion.

Contents

1	Scope and Signal Model	4
1.1	Supported modulation modes	4
2	Complex16 File Loading and IQ Normalization	4
2.1	Byte offset for a requested IQ sample	4
2.2	Signed integer to complex floating-point conversion	5
3	Fixed Samples-per-Symbol Views	5
3.1	Trimming to complete symbol blocks	5
3.2	Complex symbol averaging	6
3.3	Mean symbol power	6
4	Normalization Operations	6
4.1	Complex RMS normalization	6
4.2	Real min-max normalization	7
5	OOK/ASK Symbol-Power Slicer	7
5.1	Midrange threshold	7
6	RAW OOK Pulse Decoding	8
6.1	Envelope power	8
6.2	Rectangular moving-average filter	8
6.3	Percentile-based robust thresholding	9
6.4	Run-length encoding	9
6.5	Merging short threshold fragments	10
6.6	PWM high/low pair decision	11

6.7	Slot-quantized fallback	11
7	Complex Phase Discriminator	12
7.1	Differential phase calculation	12
7.2	Relationship to instantaneous frequency	12
8	SSTV VIS Detection	12
8.1	FM-demodulated audio and decimation	13
8.2	Tone correlation power and normalized score	13
8.3	VIS header timing search	14
8.4	VIS data, parity, and stop-tone decisions	15
9	Symbol Averaging of Real DSP Features	15
10	2-FSK, GFSK, and AFSK Path	16
10.1	2-FSK median slicing	16
10.2	Additional GFSK smoothing	16
10.3	AFSK dispatch behavior	16
11	One-Dimensional K-Means Clustering	17
11.1	Cluster initialization and assignment	17
12	4-FSK Demodulation	17
13	Ideal PSK and QAM Constellations	18
13.1	Constellation definitions	18
13.2	Constellation power normalization	19
14	PSK/QAM Constellation Demodulation	19
14.1	Optional received-symbol normalization	19
14.2	GNU Radio BPSK/QPSK path	19
14.3	Euclidean nearest-neighbor fallback	20
15	Symbol Label to Bitstream Conversion	20
15.1	Fixed-width MSB-first extraction	20
16	Top-Level Decode Dispatch	20
17	End-to-End Mathematical Paths	22
17.1	OOK symbol slicer	22
17.2	2-FSK / GFSK / AFSK	22
17.3	SSTV VIS	22
17.4	4-FSK	23
17.5	PSK/QAM	23
18	Worked DSP Examples	23
18.1	OOK symbol-power example	23
18.2	RAW PWM example	23
18.3	2-FSK phase-discriminator example	24
18.4	SSTV VIS timing and tone-score example	24
18.5	QPSK nearest-point example	24

19 Engineering Interpretation and Assumptions	25
19.1 Fixed timing is an input assumption	25
19.2 SSTV VIS uses a 30 ms timing interpretation	25
19.3 No explicit carrier recovery in the constellation fallback	25
19.4 Differential phase naturally suppresses amplitude scale	25
19.5 Median FSK threshold adapts to common offset	25
19.6 K-means estimates tone levels from the recording	25
19.7 Constellation labeling is implementation-defined by point order	26
20 DSP Function Reference Summary	26
21 Core DSP Source Listings	27
21.1 Complex16 normalization and symbol statistics	27
21.2 Normalization	27
21.3 RAW OOK DSP	27
21.4 SSTV VIS DSP	27
21.5 FSK-family DSP	28
21.6 4-FSK clustering	28
21.7 PSK/QAM decisions	28

1 Scope and Signal Model

The helper reads interleaved signed 16-bit IQ samples from a RetroSpectrum recording and converts each stored pair into a normalized complex sample

$$x[n] = I_f[n] + jQ_f[n], \quad (1)$$

where

$$I_f[n] = \frac{I_{16}[n]}{32768}, \quad Q_f[n] = \frac{Q_{16}[n]}{32768}. \quad (2)$$

For the fixed-timing digital modes, the caller supplies a samples-per-symbol value

$$N_s = \text{sps}, \quad (3)$$

and the helper groups or averages samples in fixed blocks of that length. For SSTV VIS, the same argument represents the number of IQ samples in one 30 ms VIS bit, so the input sampling rate used by that path is

$$F_{\text{in}} = \frac{\text{sps}}{0.030}. \quad (4)$$

The active processing paths are

complex16 IQ → fixed symbol blocks → OOK/PSK/QAM decisions,
 complex16 IQ → envelope → RAW OOK run lengths,
 complex16 IQ → phase difference → 2-FSK/GFSK/AFSK or 4-FSK decisions,
 complex16 IQ → phase difference → FM audio → SSTV VIS tone decisions.

1.1 Supported modulation modes

Relevant code (gnuradio_decode_file.py lines 37–50):

```

37 MODES = [
38     "ook",
39     "ook_raw",
40     "bpsk",
41     "qpsk",
42     "psk8",
43     "fsk2",
44     "gfsk",
45     "fsk4",
46     "afsk",
47     "qam16",
48     "qam64",
49     "sstv_vis",
50 ]

```

The modes share lower-level DSP primitives instead of each implementing a completely independent signal chain. In particular, FSK-family modes share the phase discriminator, PSK/QAM modes share symbol averaging and constellation decisions, both OOK modes operate on signal power, and SSTV VIS uses the phase discriminator as an FM-demodulation stage before timed audio-tone measurements.

2 Complex16 File Loading and IQ Normalization

2.1 Byte offset for a requested IQ sample

Relevant code (gnuradio_decode_file.py lines 53–77):

```

53 def read_complex16_raw(path: Path, start_sample: int, count_iq: int) -> np.ndarray:
54     if start_sample < 0:
55         raise ValueError("start-sample must be >= 0")
56     if count_iq <= 0:
57         raise ValueError("sample count must be > 0")
58     with path.open("rb") as fp:
59         fp.seek(start_sample * 4, 0)
60         raw = np.fromfile(fp, dtype=np.int16, count=count_iq * 2)
61     if raw.size % 2:
62         raw = raw[:-1]
63     iq = raw.reshape((-1, 2)).astype(np.float32)
64     return ((iq[:, 0] + 1j * iq[:, 1]) / 32768.0).astype(np.complex64)

```

One complex16 IQ sample contains two signed 16-bit integers:

$$\underbrace{I_{16}}_{2 \text{ bytes}} \quad \underbrace{Q_{16}}_{2 \text{ bytes}} . \quad (5)$$

Therefore each IQ sample occupies four bytes, and the byte offset for IQ index n_0 is

$$B_{\text{offset}} = 4n_0. \quad (6)$$

The file read requests twice as many scalar integers as complex samples because each complex sample contains an I and a Q component:

$$N_{\text{int16}} = 2N_{\text{IQ}}. \quad (7)$$

If an odd scalar count is returned, the unmatched final scalar is removed so the data can still be reshaped into complete (I, Q) pairs.

2.2 Signed integer to complex floating-point conversion

The final conversion is

$$x[n] = \frac{I_{16}[n] + jQ_{16}[n]}{32768}. \quad (8)$$

Because a signed 16-bit integer has the range

$$-32768 \leq I_{16}, Q_{16} \leq 32767, \quad (9)$$

the normalized components lie approximately in

$$-1 \leq I_f, Q_f < 1. \quad (10)$$

The divisor is 32768, rather than 32767, so the most-negative representable value maps exactly to -1.0 . The positive maximum maps to $32767/32768 \approx 0.9999695$.

3 Fixed Samples-per-Symbol Views

3.1 Trimming to complete symbol blocks

Relevant code (gnuradio_decode_file.py lines 80–107):

```

80 def symbol_views(path: Path, start_sample: int, max_symbols: int, sps: int):
81     if sps <= 0:
82         raise ValueError("samples/symbol must be > 0")
83     if max_symbols <= 0:
84         raise ValueError("max-symbols must be > 0")
85     c = read_complex16_raw(path, start_sample, max_symbols * sps)
86     usable = (c.size // sps) * sps

```

```

102 c = c[:usable]
103 blocks_view = c.reshape((-1, sps))
104 complex_symbols = blocks_view.mean(axis=1).astype(np.complex64)
105 mag_symbols = np.mean(np.abs(blocks_view) ** 2, axis=1).astype(np.float32)
107 return complex_symbols, mag_symbols, c

```

If N IQ samples are available and the supplied samples-per-symbol value is N_s , the usable count is

$$N_u = \left\lfloor \frac{N}{N_s} \right\rfloor N_s. \quad (11)$$

This discards any incomplete tail symbol. The trimmed sequence is then reshaped into a matrix

$$X[k, m] = x[kN_s + m], \quad (12)$$

where k is symbol index and $m \in \{0, \dots, N_s - 1\}$ is sample position inside the symbol.

3.2 Complex symbol averaging

The complex representative for symbol k is

$$\bar{x}_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} X[k, m]. \quad (13)$$

This is a rectangular integrate-and-dump operation. It reduces random sample noise when the symbol is approximately constant over the supplied timing interval. It also preserves the average phase and amplitude required by the PSK/QAM decision path.

3.3 Mean symbol power

For OOK/ASK, the code separately calculates

$$P_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} |X[k, m]|^2. \quad (14)$$

Since

$$|x|^2 = I^2 + Q^2, \quad (15)$$

P_k is the average complex-envelope power over symbol k . This removes carrier phase from the OOK decision variable.

4 Normalization Operations

4.1 Complex RMS normalization

Relevant code (gnuradio_decode_file.py lines 110–120):

```

110 def normalize_complex(x: np.ndarray) -> np.ndarray:
111     if x.size == 0:
112         return x.astype(np.complex64)
113     p = float(np.mean(np.abs(x) ** 2))
114     if p <= 0.0 or not math.isfinite(p):
115         return x.astype(np.complex64)
120     return (x / math.sqrt(p)).astype(np.complex64)

```

The estimated average power is

$$P = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2. \quad (16)$$

The RMS complex magnitude is $\sqrt{P}$, so the normalized samples are

$$x_N[n] = \frac{x[n]}{\sqrt{P}}. \quad (17)$$

Then, for valid nonzero P ,

$$\frac{1}{N} \sum_n |x_N[n]|^2 = 1. \quad (18)$$

This removes overall amplitude scale while preserving every sample's phase and relative amplitude.

4.2 Real min-max normalization

Relevant code (gnuradio_decode_file.py lines 123–134):

```

123 def normalize_float(x: np.ndarray) -> np.ndarray:
124     if x.size == 0:
125         return x.astype(np.float32)
126     lo = float(np.min(x))
127     hi = float(np.max(x))
128     if not math.isfinite(lo) or not math.isfinite(hi) or hi <= lo:
129         return x.astype(np.float32)
130     return ((x - lo) / (hi - lo)).astype(np.float32)

```

For a real-valued sequence with minimum $x_{\min}$ and maximum $x_{\max}$, the code applies

$$y[n] = \frac{x[n] - x_{\min}}{x_{\max} - x_{\min}}. \quad (19)$$

Thus

$$\min y = 0, \quad \max y = 1. \quad (20)$$

This is used by the OOK paths so thresholds can operate independently of absolute received power when normalization is enabled.

5 OOK/ASK Symbol-Power Slicer

5.1 Midrange threshold

Relevant code (gnuradio_decode_file.py lines 137–166):

```

137 def run_ook_symbol_slicer(mag_symbols: np.ndarray, normalize: bool) -> list[int]:
138     x = normalize_float(mag_symbols) if normalize else mag_symbols.astype(np.float32)
139     lo = float(np.min(x))
140     hi = float(np.max(x))
141     if hi <= lo:
142         return []
143     threshold = lo + 0.5 * (hi - lo)
144     tb = gr.top_block()
145     src = blocks.vector_source_f(x.tolist(), False)
146     thr = blocks.threshold_ff(threshold, threshold, 0)
147     f2b = blocks.float_to_uchar()
148     sink = blocks.vector_sink_b()
149     tb.connect(src, thr)
150     tb.connect(thr, f2b)
151     tb.connect(f2b, sink)
152     tb.run()
153     return [int(v) & 1 for v in sink.data()]

```

The threshold is the midpoint between the observed low and high values:

$$T = x_{\min} + \frac{1}{2}(x_{\max} - x_{\min}) \quad (21)$$

$$= \frac{x_{\min} + x_{\max}}{2}. \quad (22)$$

The intended binary decision is

$$b_k = \begin{cases} 1, & P_k \geq T, \\ 0, & P_k < T. \end{cases} \quad (23)$$

When min-max normalization is enabled, $x_{\min} = 0$ and $x_{\max} = 1$, so the threshold becomes exactly

$$T = 0.5. \quad (24)$$

The GNU Radio `threshold_ff` block performs the threshold operation, `float_to_uchar` converts its output to bytes, and the final bit mask `& 1` constrains each recovered value to one bit.

6 RAW OOK Pulse Decoding

The RAW OOK path does not assume that one supplied block directly corresponds to one binary bit. Instead, it detects high/low envelope runs and interprets relative pulse widths.

6.1 Envelope power

Relevant code (`gnuradio_decode_file.py` lines 276–307):

```

276 def run_ook_raw_pulse_decoder(samples: np.ndarray, sps: int, normalize: bool) -> list[int]:
295     if samples.size < 2 or sps <= 0:
296         return []
298     env = (np.abs(samples) ** 2).astype(np.float32)
299     smooth_len = max(1, min(max(1, sps // 20), 128))
300     env_smooth = moving_average(env, smooth_len)
301     env_smooth = normalize_float(env_smooth) if normalize else env_smooth
304     states = threshold_binary_robust(env_smooth)
305     runs = build_runs_from_states(states)
306     runs = merge_short_runs(runs, max(1, sps // 10))
307     bits = decode_pwm_pairs_from_runs(runs, max(1, sps // 5))

```

The envelope variable is actually instantaneous complex-envelope power:

$$E[n] = |x[n]|^2 = I[n]^2 + Q[n]^2. \quad (25)$$

The smoothing length is

$$N_{\text{smooth}} = \max \left(1, \min \left(\max \left(1, \left\lfloor \frac{N_s}{20} \right\rfloor \right), 128 \right) \right). \quad (26)$$

For normal positive N_s , this is approximately $N_s/20$ samples, capped at 128. It suppresses very short envelope spikes while keeping the smoothing window much shorter than the caller's base timing unit.

6.2 Rectangular moving-average filter

Relevant code (`gnuradio_decode_file.py` lines 478–487):

```

478 def moving_average(x: np.ndarray, n: int) -> np.ndarray:
483     if x.size == 0 or n <= 1:
484         return x.astype(np.float32)
485     n = min(n, x.size)
486     kernel = np.ones(n, dtype=np.float32) / float(n)
487     return np.convolve(x, kernel, mode="same").astype(np.float32)

```

The impulse response of this FIR smoother is

$$h[m] = \begin{cases} \frac{1}{N}, & 0 \leq m < N, \\ 0, & \text{otherwise.} \end{cases} \quad (27)$$

The convolution is

$$y[n] = (x * h)[n] = \frac{1}{N} \sum_{m=0}^{N-1} x[n-m], \quad (28)$$

subject to NumPy's centered `mode="same"` boundary behavior. Its discrete-time frequency response is

$$H(e^{j\omega}) = \frac{1}{N} e^{-j\omega(N-1)/2} \frac{\sin(N\omega/2)}{\sin(\omega/2)}. \quad (29)$$

Thus it is a rectangular-window low-pass smoother: low-frequency envelope or discriminator trends are preserved while rapid sample-to-sample variations are attenuated.

6.3 Percentile-based robust thresholding

Relevant code (gnuradio_decode_file.py lines 218–244):

```

218 def threshold_binary_robust(x: np.ndarray) -> np.ndarray:
219     vals = x.astype(np.float32)
220     for low_pct, high_pct in ((5, 95), (10, 90), (15, 85), (20, 80), (25, 75)):
221         lo = float(np.percentile(vals, low_pct))
222         hi = float(np.percentile(vals, high_pct))
223         if math.isfinite(lo) and math.isfinite(hi) and hi > lo:
224             threshold = lo + 0.5 * (hi - lo)
225             states = (vals >= threshold).astype(np.uint8)
226             ones = int(np.count_nonzero(states))
227             if 0 < ones < states.size and np.count_nonzero(np.diff(states)) > 0:
228                 return states
229     lo = float(np.min(vals))
230     hi = float(np.max(vals))
231     if math.isfinite(lo) and math.isfinite(hi) and hi > lo:
232         threshold = lo + 0.5 * (hi - lo)
233     return (vals >= threshold).astype(np.uint8)

```

For each percentile pair (p_L, p_H) , the code estimates

$$L = Q_{p_L}(x), \quad H = Q_{p_H}(x), \quad (30)$$

then uses

$$T = \frac{L + H}{2}. \quad (31)$$

The binary state sequence is

$$s[n] = \mathbf{1}\{x[n] \geq T\}. \quad (32)$$

The sequence is accepted only if it contains both states and at least one transition. The search starts with the outer 5th/95th percentiles and progressively moves inward to 25th/75th percentiles. Compared with raw minimum/maximum thresholding, percentiles reduce the influence of isolated extreme samples. If every percentile attempt fails, the code falls back to the ordinary midrange threshold.

6.4 Run-length encoding

Relevant code (gnuradio_decode_file.py lines 195–215):

```

195 def build_runs_from_states(states: np.ndarray) -> list[tuple[int, int]]:
203     runs: list[tuple[int, int]] = []
204     cur = int(states[0])
205     count = 1
206     for val in states[1:]:
207         v = int(val)
208         if v == cur:
209             count += 1
210         else:
211             runs.append((cur, count))
212             cur = v
213             count = 1
214     runs.append((cur, count))
215     return runs

```

A contiguous region with state $a_r \in \{0, 1\}$ and length L_r is represented as the pair

$$R_r = (a_r, L_r). \quad (33)$$

This converts the thresholded sample stream into a pulse-duration representation. For a run spanning sample indices n_a through n_b inclusive,

$$L_r = n_b - n_a + 1. \quad (34)$$

If the true sampling rate is F_s , the physical run duration would be

$$T_r = \frac{L_r}{F_s}, \quad (35)$$

but this helper intentionally works in sample or slot units because it is given `sps`, not F_s .

6.5 Merging short threshold fragments

Relevant code (gnuradio_decode_file.py lines 169–192):

```

169 def merge_short_runs(runs: list[tuple[int, int]], min_len: int) -> list[tuple[int, int]]:
177     out: list[tuple[int, int]] = []
178     for state, length in runs:
179         if length < min_len and out:
180             prev_state, prev_len = out[-1]
181             out[-1] = (prev_state, prev_len + length)
182         else:
183             out.append((state, length))
184     merged: list[tuple[int, int]] = []
185     for state, length in out:
186         if merged and merged[-1][0] == state:
187             prev_state, prev_len = merged[-1]
188             merged[-1] = (prev_state, prev_len + length)
189         else:
190             merged.append((state, length))
191     return merged

```

For any noninitial run satisfying

$$L_r < L_{\min}, \quad (36)$$

the implementation absorbs its length into the preceding run. Adjacent runs that then have the same state are coalesced. In the direct RAW path,

$$L_{\min, \text{merge}} = \max \left(1, \left\lceil \frac{N_s}{10} \right\rceil \right). \quad (37)$$

The operation is a heuristic deglitcher: very short state flips are interpreted as threshold fragmentation rather than intentional pulse structure.

6.6 PWM high/low pair decision

Relevant code (gnuradio_decode_file.py lines 248–273):

```

248 def decode_pwm_pairs_from_runs(runs: list[tuple[int, int]], min_len: int) -> list[int]:
255     bits: list[int] = []
257     min_len = max(1, min_len)
259     while i + 1 < len(runs):
260         state_a, len_a = runs[i]
261         state_b, len_b = runs[i + 1]
263         if state_a == 1 and state_b == 0:
265             if len_a >= min_len and len_b >= min_len:
266                 bits.append(1 if len_a > len_b else 0)
267                 i += 2
268                 continue
271         i += 1
273     return bits

```

For each valid high-low run pair

$$(1, L_H), (0, L_L), \quad (38)$$

the recovered bit is

$$b_i = \begin{cases} 1, & L_H > L_L, \\ 0, & L_H \leq L_L. \end{cases} \quad (39)$$

Both durations must satisfy $L_H, L_L \geq L_{\min}$. In the direct sample path,

$$L_{\min} = \max\left(1, \left\lfloor \frac{N_s}{5} \right\rfloor\right). \quad (40)$$

This is a relative-width decoder: it does not need the exact absolute pulse duration as long as the high and low durations remain distinguishable.

6.7 Slot-quantized fallback

Relevant code (gnuradio_decode_file.py lines 308–324):

```

308     if len(bits) >= 2:
309         return bits
312     usable = (env_smooth.size // sps) * sps
313     if usable <= 0:
314         return bits
316     slot_vals = env_smooth[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
317     slot_states = threshold_binary_robust(slot_vals)
318     slot_runs = build_runs_from_states(slot_states)
319     slot_runs = merge_short_runs(slot_runs, 1)
320     slot_bits = decode_pwm_pairs_from_runs(slot_runs, 1)
322     if len(slot_bits) > len(bits):
323         return slot_bits
324     return bits

```

If direct sample-run decoding produces fewer than two bits, the envelope is first averaged into blocks of N_s samples:

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} E_s[kN_s + m]. \quad (41)$$

Thresholding and run-length decoding then operate in one-unit slots rather than raw samples. The code returns whichever path produced more recovered bits. This provides a second timing interpretation when individual sample transitions are noisy or slightly misaligned.

7 Complex Phase Discriminator

7.1 Differential phase calculation

Relevant code (gnuradio_decode_file.py lines 327–337):

```

327 def phase_discriminator(samples: np.ndarray) -> np.ndarray:
334     if samples.size < 2:
335         return np.array([], dtype=np.float32)
336     x = normalize_complex(samples)
337     return np.angle(x[1:] * np.conj(x[:-1])).astype(np.float32)

```

Write a complex sample in polar form as

$$x[n] = A[n]e^{j\phi[n]}. \quad (42)$$

Then

$$x[n]x^*[n-1] = A[n]A[n-1]e^{j(\phi[n]-\phi[n-1])}. \quad (43)$$

Taking its argument gives

$$d[n] = \arg\{x[n]x^*[n-1]\} = \text{wrap}_{[-\pi,\pi)}(\phi[n] - \phi[n-1]). \quad (44)$$

Thus the returned values are phase increments in radians per sample. The preliminary unit-power normalization does not alter the phase-difference result because multiplication by a common positive real scale has zero phase.

7.2 Relationship to instantaneous frequency

For a complex sinusoid

$$x[n] = Ae^{j(2\pi fn/F_s + \phi_0)}, \quad (45)$$

its adjacent phase difference is

$$d[n] = \frac{2\pi f}{F_s} \quad (46)$$

when the increment remains inside the principal phase interval. Therefore, if F_s were supplied, frequency in Hz could be recovered by

$$f[n] = \frac{F_s}{2\pi} d[n]. \quad (47)$$

The FSK decision paths omit this final scale because they require only relative discriminator levels and therefore work directly in radians/sample. The SSTV VIS path derives an input sampling rate from the 30 ms VIS timing and retains an explicit effective audio sampling rate for its tone correlations.

8 SSTV VIS Detection

The SSTV VIS path uses the phase discriminator as an FM demodulator, converts the discriminator output to a lower-rate audio sequence, searches for the timed VIS header tones, and returns the seven VIS data decisions followed by the even-parity decision. Synchronization tones are used for detection and validation rather than emitted as bits.

8.1 FM-demodulated audio and decimation

Relevant code (gnuradio_decode_file.py lines 340–364):

```

340 def sstv_fm_audio(samples: np.ndarray, input_rate: float) -> tuple[np.ndarray, float]:
341     discriminator = phase_discriminator(samples).astype(np.float64)
342     discriminator -= np.mean(discriminator)
343     target_rate = 24000.0
344     decimation = max(1, int(math.floor(input_rate / target_rate)))
345     usable = (discriminator.size // decimation) * decimation
346     audio = discriminator[:usable].reshape((-1, decimation)).mean(axis=1)
347     audio -= np.mean(audio)
348     peak = float(np.max(np.abs(audio)))
349     if peak > 0.0 and math.isfinite(peak):
350         audio /= peak
351     return audio.astype(np.float32), input_rate / float(decimation)

```

Let the discriminator output be $d[n]$. The mean discriminator value is removed:

$$d_0[n] = d[n] - \frac{1}{N} \sum_{i=0}^{N-1} d[i]. \quad (48)$$

This removes a constant phase-increment bias before audio-tone measurements.

The integer decimation factor is

$$D = \max \left(1, \left\lfloor \frac{F_{\text{in}}}{24000} \right\rfloor \right). \quad (49)$$

After trimming to a multiple of D , each output audio sample is a block average

$$a[k] = \frac{1}{D} \sum_{m=0}^{D-1} d_0[kD + m]. \quad (50)$$

The effective audio sample rate is

$$F_a = \frac{F_{\text{in}}}{D}. \quad (51)$$

The audio mean is removed again and the sequence is divided by its peak absolute value when that peak is finite and nonzero. The resulting scale does not affect the later normalized tone-power ratios.

8.2 Tone correlation power and normalized score

Relevant code (gnuradio_decode_file.py lines 367–409):

```

367 def sstv_tone_power(samples: np.ndarray, sample_rate: float, frequency: float) -> float:
368     x = samples.astype(np.float64)
369     x -= np.mean(x)
370     x *= np.hanning(x.size)
371     n = np.arange(x.size, dtype=np.float64)
372     angle = 2.0 * math.pi * frequency * n / sample_rate
373     real = float(np.dot(x, np.cos(angle)))
374     imag = float(np.dot(x, np.sin(angle)))
375     return (real * real) + (imag * imag)
376
377 segment = audio[start:end]
378 frequencies = (1100.0, 1200.0, 1300.0, 1900.0)
379 powers = [sstv_tone_power(segment, sample_rate, f) for f in frequencies]
380 total = sum(powers) + 1e-18
381 return powers[target_index] / total

```

For a finite audio segment $a[n]$, the code removes the segment mean and applies a Hann window $w[n]$. For a candidate frequency f_0 , define

$$C(f_0) = \sum_{n=0}^{N-1} a[n]w[n]e^{-j2\pi f_0 n/F_a}. \quad (52)$$

The correlation power is

$$P(f_0) = |C(f_0)|^2 = \Re\{C(f_0)\}^2 + \Im\{C(f_0)\}^2. \quad (53)$$

The candidate set is

$$\mathcal{F} = \{1100, 1200, 1300, 1900\} \text{ Hz}, \quad (54)$$

and the normalized score for target frequency $f_t \in \mathcal{F}$ is

$$S(f_t) = \frac{P(f_t)}{\sum_{f \in \mathcal{F}} P(f) + 10^{-18}}. \quad (55)$$

Thus each score represents the fraction of measured candidate-tone power associated with the requested tone.

8.3 VIS header timing search

Relevant code (gnuradio_decode_file.py lines 412–450):

```

412 def run_sstv_vis(samples: np.ndarray, sps: int) -> list[int]:
420     # One VIS bit is 30 ms, so Samples/Symbol represents one 30 ms VIS bit.
421     input_rate = float(sps) / 0.030
422     audio, audio_rate = sstv_fm_audio(samples, input_rate)
426     header_seconds = 0.910
431     step = max(1, int(round(0.005 * audio_rate)))
436     for start in range(0, last_start + 1, step):
438         score = (
439             sstv_tone_score(audio, audio_rate, t + 0.050, 0.200, 1900.0)
440             + sstv_tone_score(audio, audio_rate, t + 0.300, 0.010, 1200.0)
441             + sstv_tone_score(audio, audio_rate, t + 0.360, 0.200, 1900.0)
442             + sstv_tone_score(audio, audio_rate, t + 0.615, 0.020, 1200.0)
443         ) / 4.0
449     if best_start is None or best_score < 0.55:
450         return []

```

The timing represented by one VIS bit is

$$T_b = 0.030 \text{ s}, \quad (56)$$

so the input rate is derived from the supplied sample count as

$$F_{\text{in}} = \frac{N_s}{T_b}. \quad (57)$$

The complete header interval used by the decoder is

$$0.300 + 0.010 + 0.300 + 0.030 + 7(0.030) + 0.030 + 0.030 = 0.910 \text{ s}. \quad (58)$$

Candidate starts are examined every 5 ms. For candidate start time t , the header score is

$$H(t) = \frac{1}{4} (S_{1900}(t + 0.050, 0.200) + S_{1200}(t + 0.300, 0.010) \quad (59)$$

$$+ S_{1900}(t + 0.360, 0.200) + S_{1200}(t + 0.615, 0.020)), \quad (60)$$

where $S_f(t_0, T)$ denotes the normalized score for frequency f over a window beginning at t_0 with duration T . The highest-scoring candidate is accepted when

$$H_{\text{max}} \geq 0.55. \quad (61)$$

8.4 VIS data, parity, and stop-tone decisions

Relevant code (gnuradio_decode_file.py lines 452–475):

```

452 bit_start = best_start + 0.640
453 for bit_index in range(7):
454     t = bit_start + (float(bit_index) * 0.030) + 0.005
455     one_score = sstv_tone_score(audio, audio_rate, t, 0.020, 1100.0)
456     zero_score = sstv_tone_score(audio, audio_rate, t, 0.020, 1300.0)
457     bits.append(1 if one_score > zero_score else 0)
458 parity_time = bit_start + (7.0 * 0.030) + 0.005
459 parity_one = sstv_tone_score(audio, audio_rate, parity_time, 0.020, 1100.0)
460 parity_zero = sstv_tone_score(audio, audio_rate, parity_time, 0.020, 1300.0)
461 parity = 1 if parity_one > parity_zero else 0
462 stop_time = bit_start + (8.0 * 0.030) + 0.005
463 stop_score = sstv_tone_score(audio, audio_rate, stop_time, 0.020, 1200.0)
464 if ((sum(bits) + parity) & 1) != 0:
465     if stop_score < 0.45:
466         bits.append(parity)
467     return bits

```

The seven data intervals begin at $t_{\text{VIS}} = t_{\text{header}} + 0.640$ s. Within each 30 ms interval, the decoder measures a centered 20 ms region beginning 5 ms into the interval. The data decision is

$$b_i = \begin{cases} 1, & S_{1100} > S_{1300}, \\ 0, & S_{1100} \leq S_{1300}. \end{cases} \quad (62)$$

The parity interval uses the same 1100/1300 Hz comparison. The result is accepted only when

$$\left(\sum_{i=0}^6 b_i + p \right) \bmod 2 = 0 \quad (63)$$

and the 1200 Hz stop-tone score satisfies

$$S_{1200, \text{stop}} \geq 0.45. \quad (64)$$

The output from this mode is the seven transmitted VIS data bits followed by the parity bit. The leader, break, start, and stop tones are detected as timing or validation features and are not emitted as binary output. The subsequent SSTV image scan waveform is not converted into bits by this helper.

9 Symbol Averaging of Real DSP Features

Relevant code (gnuradio_decode_file.py lines 490–501):

```

490 def symbol_average_float(x: np.ndarray, sps: int, max_symbols: int) -> np.ndarray:
491     if x.size == 0 or sps <= 0:
492         return np.array([], dtype=np.float32)
493     usable = (x.size // sps) * sps
494     if usable <= 0:
495         return np.array([], dtype=np.float32)
496     vals = x[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
497     return vals[:max_symbols]

```

For a real-valued feature stream $d[n]$, such as phase-discriminator output, the symbol statistic is

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} d[kN_s + m]. \quad (65)$$

This integrates the feature over one assumed symbol period, reducing zero-mean sample noise and producing one decision variable per symbol.

10 2-FSK, GFSK, and AFSK Path

10.1 2-FSK median slicing

Relevant code (gnuradio_decode_file.py lines 504–518):

```

504 def run_fsk2(samples: np.ndarray, sps: int, max_symbols: int, gfsk: bool = False) -> list[int]:
511     freq = phase_discriminator(samples)
512     if gfsk:
513         freq = moving_average(freq, max(3, sps // 3))
514     vals = symbol_average_float(freq, sps, max_symbols)
515     if vals.size == 0:
516         return []
517     threshold = float(np.median(vals))
518     return [1 if v >= threshold else 0 for v in vals]

```

For ordinary 2-FSK, let $d[n]$ be the differential phase. The symbol statistic is

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} d[kN_s + m]. \quad (66)$$

The slicing threshold is the sample median of all symbol statistics:

$$T = \text{median}\{v_k\}. \quad (67)$$

The bit decision is

$$b_k = \begin{cases} 1, & v_k \geq T, \\ 0, & v_k < T. \end{cases} \quad (68)$$

For two approximately balanced FSK tones, the median usually lies between the two discriminator clusters. Unlike a fixed zero threshold, this can tolerate a common residual frequency offset that shifts both clusters together.

10.2 Additional GFSK smoothing

When `gfsk=True`, the discriminator is first low-pass smoothed with

$$N_G = \max\left(3, \left\lfloor \frac{N_s}{3} \right\rfloor\right). \quad (69)$$

Thus

$$d_G[n] = \frac{1}{N_G} \sum_{m=0}^{N_G-1} d[n - m]. \quad (70)$$

GFSK intentionally has smoother frequency transitions than rectangular-pulse FSK, so this extra averaging reduces high-frequency discriminator variation before the per-symbol integration stage.

10.3 AFSK dispatch behavior

The dispatch logic sends `afsk` through the same function as 2-FSK, with `gfsk=False`. Therefore the implemented AFSK mathematics in this helper is exactly the same differential-phase, symbol-average, and median-slice chain shown above. There is no separate mark/space matched-filter bank in this source.

11 One-Dimensional K-Means Clustering

11.1 Cluster initialization and assignment

Relevant code (gnuradio_decode_file.py lines 521–545):

```

521 def kmeans_1d(vals: np.ndarray, k: int, rounds: int = 24) -> np.ndarray:
522     if vals.size < k:
523         lo = float(np.min(vals))
524         hi = float(np.max(vals))
525         return np.linspace(lo, hi, k, dtype=np.float32)
526     centers = np.percentile(vals, np.linspace(5, 95, k)).astype(np.float32)
527     for _ in range(rounds):
528         dist = np.abs(vals[:, None] - centers[None, :])
529         labels = np.argmin(dist, axis=1)
530         new_centers = centers.copy()
531         for idx in range(k):
532             part = vals[labels == idx]
533             if part.size:
534                 new_centers[idx] = float(np.mean(part))
535         if np.allclose(new_centers, centers):
536             break
537         centers = new_centers
538     return np.sort(centers)

```

For K clusters with centers c_j , each observation is assigned to its nearest center:

$$\ell_i = \arg \min_{0 \leq j < K} |v_i - c_j|. \quad (71)$$

Each nonempty center is then updated to the mean of its assigned values:

$$c_j^{\text{new}} = \frac{1}{|S_j|} \sum_{v_i \in S_j} v_i. \quad (72)$$

This alternates for at most 24 rounds or until `np.allclose` determines that the centers have effectively stopped moving.

Initialization uses evenly spaced percentiles between 5 and 95:

$$c_j^{(0)} = Q_{p_j}(v), \quad p_j \in \text{linspace}(5, 95, K). \quad (73)$$

Percentile initialization reduces sensitivity to single extreme discriminator samples. If fewer observations than clusters are available, the fallback places K centers evenly between the observed minimum and maximum.

12 4-FSK Demodulation

Relevant code (gnuradio_decode_file.py lines 548–560):

```

548 def run_fsk4(samples: np.ndarray, sps: int, max_symbols: int) -> list[int]:
549     vals = symbol_average_float(phase_discriminator(samples), sps, max_symbols)
550     if vals.size == 0:
551         return []
552     centers = kmeans_1d(vals, 4)
553     labels = np.argmin(np.abs(vals[:, None] - centers[None, :]), axis=1)
554     return [int(v) & 0x3 for v in labels]

```

The first two stages are the same differential-phase and per-symbol averaging used for 2-FSK:

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} \arg\{x[n]x^*[n-1]\}. \quad (74)$$

Instead of one threshold, the values are clustered into four ordered centers

$$c_0 \leq c_1 \leq c_2 \leq c_3. \quad (75)$$

Each symbol label is

$$s_k = \arg \min_{j \in \{0,1,2,3\}} |v_k - c_j|. \quad (76)$$

Because the centers are sorted before labeling, increasing symbol indices correspond to increasing discriminator level. The final & 0x3 restricts the integer label to the range 0–3.

13 Ideal PSK and QAM Constellations

13.1 Constellation definitions

Relevant code (gnuradio_decode_file.py lines 563–582):

```

563 def constellation_points(mod: str) -> tuple[np.ndarray, int]:
564     if mod == "bpsk":
565         return np.array([-1 + 0j, 1 + 0j], dtype=np.complex64), 1
566     if mod == "qpsk":
567         return np.array([1 + 1j, -1 + 1j, -1 - 1j, 1 - 1j], dtype=np.complex64), 2
568     if mod == "psk8":
569         pts = [np.exp(1j * 2.0 * math.pi * n / 8.0) for n in range(8)]
570         return np.array(pts, dtype=np.complex64), 3
571     if mod == "qam16":
572         levels = [-3, -1, 1, 3]
573         pts = [complex(i, q) for q in levels for i in levels]
574         return np.array(pts, dtype=np.complex64), 4
575     if mod == "qam64":
576         levels = [-7, -5, -3, -1, 1, 3, 5, 7]
577         pts = [complex(i, q) for q in levels for i in levels]
578         return np.array(pts, dtype=np.complex64), 6

```

For BPSK, the ideal set is

$$\mathcal{C}_{\text{BPSK}} = \{-1, +1\}, \quad (77)$$

with one bit per symbol.

For QPSK, the source uses the four points

$$\mathcal{C}_{\text{QPSK}} = \{1 + j, -1 + j, -1 - j, 1 - j\}, \quad (78)$$

with two bits per symbol.

For 8PSK,

$$p_m = e^{j2\pi m/8}, \quad m = 0, \dots, 7, \quad (79)$$

so all eight points lie on the unit circle and carry three bits per symbol.

For square M -QAM, each point is

$$p = I + jQ, \quad (80)$$

with I and Q drawn from equally spaced odd-integer levels. For 16QAM,

$$I, Q \in \{-3, -1, 1, 3\}, \quad (81)$$

and for 64QAM,

$$I, Q \in \{-7, -5, -3, -1, 1, 3, 5, 7\}. \quad (82)$$

The list order defines the fallback integer symbol labels; this source does not apply a separate Gray-code remapping table to these fallback QAM labels.

13.2 Constellation power normalization

Before the NumPy fallback computes distances, the ideal constellation itself is passed through `normalize_complex`. Therefore the reference points have unit average power:

$$\frac{1}{M} \sum_{m=0}^{M-1} |p'_m|^2 = 1. \quad (83)$$

For example, the unnormalized average energy of square 16QAM with levels $\{-3, -1, 1, 3\}$ is

$$E_s = 10, \quad (84)$$

so normalization scales its coordinates by $1/\sqrt{10}$.

14 PSK/QAM Constellation Demodulation

14.1 Optional received-symbol normalization

Relevant code (gnuradio_decode_file.py lines 586–623):

```

586 def run_constellation_demod(symbols: np.ndarray, mod: str, normalize: bool) -> tuple[list[int], int]:
593     if symbols.size == 0:
594         return [], 1
596     x = normalize_complex(symbols) if normalize else symbols.astype(np.complex64)
601     if mod in {"bpsk", "qpsk"}:
602         try:
603             if mod == "bpsk" and hasattr(digital, "constellation_bpsk"):
604                 const = digital.constellation_bpsk().base()
605             elif mod == "qpsk" and hasattr(digital, "constellation_qpsk"):
606                 const = digital.constellation_qpsk().base()
609             tb = gr.top_block()
610             src = blocks.vector_source_c(x.tolist(), False)
611             dec = digital.constellation_decoder_cb(const)
612             sink = blocks.vector_sink_b()
613             tb.connect(src, dec)
614             tb.connect(dec, sink)
615             tb.run()
616             return [int(v) for v in sink.data()], (1 if mod == "bpsk" else 2)
620     pts, bps = constellation_points(mod)
621     pts = normalize_complex(pts)
622     labels = np.argmax(np.abs(x[:, None] - pts[None, :]), axis=1)
623     return [int(v) for v in labels], bps

```

When normalization is requested, the received symbol sequence is scaled to unit average power:

$$r'_k = \frac{r_k}{\sqrt{\frac{1}{K} \sum_i |r_i|^2}}. \quad (85)$$

This makes its average scale comparable with the normalized ideal constellation. It does not estimate or remove carrier phase rotation, residual frequency rotation, or symbol-timing error; those conditions must already be sufficiently controlled for nearest-point decoding to work.

14.2 GNU Radio BPSK/QPSK path

For BPSK and QPSK, the script first attempts to use GNU Radio's built-in constellation objects and `constellation_decoder_cb`. Conceptually, a constellation decoder assigns each received symbol to a constellation decision region and outputs the associated symbol index. If the convenience objects are unavailable or the block raises an exception, the code moves to the generic NumPy nearest-point path.

14.3 Euclidean nearest-neighbor fallback

For every received complex symbol r_k and every normalized ideal point p_j , the code calculates the Euclidean complex-plane distance

$$d_{k,j} = |r_k - p_j|. \quad (86)$$

Equivalently,

$$d_{k,j}^2 = (\Re\{r_k - p_j\})^2 + (\Im\{r_k - p_j\})^2. \quad (87)$$

The selected symbol is

$$\hat{s}_k = \arg \min_j d_{k,j}. \quad (88)$$

Because the square root is monotonic, minimizing $d_{k,j}$ is equivalent to minimizing squared Euclidean distance. The NumPy expression uses the magnitude directly.

This is the standard minimum-distance decision rule for equally likely constellation points in isotropic additive white Gaussian noise when the constellation is correctly aligned and scaled.

15 Symbol Label to Bitstream Conversion

15.1 Fixed-width MSB-first extraction

Relevant code (gnuradio_decode_file.py lines 626–648):

```

626 def symbol_to_bits(symbol: int, bps: int, invert: bool) -> str:
631     out = []
632     for bit_index in range(bps - 1, -1, -1):
633         bit = (symbol >> bit_index) & 1
634         if invert:
635             bit ^= 1
636         out.append("1" if bit else "0")
637     return "".join(out)
640 def write_bits(symbols: Iterable[int], bps: int, invert: bool, tight: bool) -> None:
645     sep = "" if tight else " "
646     parts = [symbol_to_bits(int(sym), bps, invert) for sym in symbols]
647     sys.stdout.write(sep.join(parts))
648     sys.stdout.write("\n")

```

For a symbol integer s and B bits per symbol, the bit at position q is extracted by

$$b_q = (s \gg q) \& 1. \quad (89)$$

The loop runs from $q = B - 1$ down to 0, so output is most-significant-bit first. If inversion is enabled,

$$b'_q = b_q \oplus 1 = 1 - b_q. \quad (90)$$

Thus, for example, with $B = 2$,

$$s = 2 = (10)_2 \rightarrow 10, \quad (91)$$

and inversion yields 01.

16 Top-Level Decode Dispatch

Relevant code (gnuradio_decode_file.py lines 676–738):

```

676     if args.mod == "sstv_vis":
677         # For SSTV VIS, Samples/Symbol is the number of IQ samples in one 30 ms VIS bit.
678         input_rate = float(args.sps) / 0.030
679         search_samples = int(math.ceil(input_rate * 5.0))
682         raw_samples = read_complex16_raw(path, args.start_sample, search_samples)

```

```

691     symbols = run_sstv_vis(raw_samples, args.sps)
692     if not symbols:
693         print("No valid SSTV VIS header was detected.", file=sys.stderr)
696     write_bits(symbols, 1, bool(args.invert), bool(args.tight))
697     return 0
699     try:
700         complex_symbols, mag_symbols, raw_samples = symbol_views(
701             path, args.start_sample, args.max_symbols, args.sps
715         if args.mod == "ook":
716             symbols = run_ook_symbol_slicer(mag_symbols, bool(args.normalize))
718         elif args.mod == "ook_raw":
719             symbols = run_ook_raw_pulse_decoder(raw_samples, args.sps, bool(args.normalize))
721         elif args.mod in {"fsk2", "gfsk", "afsk"}:
722             symbols = run_fsk2(raw_samples, args.sps, args.max_symbols, gfsk=(args.mod == "gfsk"))
724         elif args.mod == "fsk4":
725             symbols = run_fsk4(raw_samples, args.sps, args.max_symbols)
727         elif args.mod in {"bpsk", "qpsk", "psk8", "qam16", "qam64"}:
728             symbols, natural_bps = run_constellation_demod(complex_symbols, args.mod, bool(args.normalize))
737         bps = user_bps if 1 <= user_bps <= 8 else natural_bps
738         write_bits(symbols, bps, invert, tight)

```

The SSTV VIS path is dispatched before the ordinary fixed-symbol views are built. It derives the input rate from the supplied 30 ms VIS sample count, reads a five-second IQ search interval beginning at the requested start sample, performs VIS detection, and writes one bit per recovered VIS decision. The remaining modes use the fixed-symbol or raw-sample paths described earlier.

The dispatch structure shows which intermediate representation feeds each modulation family:

Mode	Input statistic	Decision mechanism
OOK	mean symbol power P_k	midrange threshold through GNU Radio
OOK RAW	raw envelope power $E[n]$	robust threshold + run-length PWM comparison
SSTV VIS	FM-demodulated audio	timed 1900/1200 Hz header scoring; 1100/1300 Hz VIS decisions
2-FSK / AFSK	differential phase $d[n]$	symbol mean + median threshold
GFSK	smoothed differential phase	symbol mean + median threshold
4-FSK	differential phase	symbol mean + 4-center k-means
BPSK / QPSK	complex symbol average $\bar{x}_k$	GNU Radio decoder when available; otherwise nearest point
8PSK / 16QAM / 64QAM	complex symbol average $\bar{x}_k$	Euclidean nearest ideal point

For the ordinary symbol-label modes, the output bit width is selected by

$$B = \begin{cases} B_{\text{user}}, & 1 \leq B_{\text{user}} \leq 8, \\ B_{\text{natural}}, & \text{otherwise.} \end{cases} \quad (92)$$

The SSTV VIS path writes its recovered decisions with a fixed width of one bit per returned value.

17 End-to-End Mathematical Paths

17.1 OOK symbol slicer

Starting from complex16 storage,

$$x[n] = \frac{I_{16}[n] + jQ_{16}[n]}{32768}, \quad (93)$$

$$P_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} |x[kN_s + m]|^2, \quad (94)$$

$$T = \frac{\min(P) + \max(P)}{2}, \quad (95)$$

$$b_k = \mathbf{1}\{P_k \geq T\}. \quad (96)$$

If normalization is enabled, P_k is first transformed to the $[0, 1]$ interval.

17.2 2-FSK / GFSK / AFSK

The core path is

$$x[n] = A[n]e^{j\phi[n]}, \quad (97)$$

$$d[n] = \arg\{x[n]x^*[n-1]\}, \quad (98)$$

$$\tilde{d}[n] = d[n] * h[n] \quad (\text{GFSK only}), \quad (99)$$

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} \tilde{d}[kN_s + m], \quad (100)$$

$$T = \text{median}\{v_k\}, \quad (101)$$

$$b_k = \mathbf{1}\{v_k \geq T\}. \quad (102)$$

For ordinary FSK2 and AFSK, $\tilde{d} = d$.

17.3 SSTV VIS

With $T_b = 0.030$ s and $N_s = \text{sps}$,

$$F_{\text{in}} = \frac{N_s}{T_b}, \quad (103)$$

$$d[n] = \arg\{x[n]x^*[n-1]\}, \quad (104)$$

$$D = \max\left(1, \left\lfloor \frac{F_{\text{in}}}{24000} \right\rfloor\right), \quad (105)$$

$$a[k] = \frac{1}{D} \sum_{m=0}^{D-1} (d[kD + m] - \bar{d}), \quad (106)$$

$$P(f) = \left| \sum_n a[n]w[n]e^{-j2\pi fn/F_a} \right|^2, \quad (107)$$

$$S(f) = \frac{P(f)}{P(1100) + P(1200) + P(1300) + P(1900) + 10^{-18}}. \quad (108)$$

The header location maximizes the average of the four timed 1900/1200 Hz scores. Each VIS data interval is then decided by comparing $S(1100)$ and $S(1300)$, followed by even-parity and 1200 Hz stop-tone validation.

17.4 4-FSK

The first three calculations are the same through v_k . Then

$$\{c_0, c_1, c_2, c_3\} = \text{kmeans}_4(\{v_k\}), \quad (109)$$

$$s_k = \arg \min_j |v_k - c_j|. \quad (110)$$

The resulting $s_k \in \{0, 1, 2, 3\}$ is normally represented by two output bits.

17.5 PSK/QAM

For fixed symbol timing,

$$\bar{x}_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} x[kN_s + m], \quad (111)$$

$$r_k = \frac{\bar{x}_k}{\sqrt{\frac{1}{K} \sum_i |\bar{x}_i|^2}} \quad (\text{if normalize}), \quad (112)$$

$$p'_j = \frac{p_j}{\sqrt{\frac{1}{M} \sum_m |p_m|^2}}, \quad (113)$$

$$\hat{s}_k = \arg \min_j |r_k - p'_j|. \quad (114)$$

For BPSK/QPSK, GNU Radio may supply the decision stage instead of the explicit final nearest-point equation.

18 Worked DSP Examples

18.1 OOK symbol-power example

Suppose four normalized symbol-power values are

$$P = \{0.08, 0.92, 0.11, 0.88\}. \quad (115)$$

Then

$$T = 0.08 + \frac{1}{2}(0.92 - 0.08) = 0.50. \quad (116)$$

The decisions are therefore

$$\{0, 1, 0, 1\}. \quad (117)$$

18.2 RAW PWM example

Assume the thresholded envelope produces the run pairs

$$(1, 30), (0, 10), (1, 10), (0, 30). \quad (118)$$

For the first pair, $30 > 10$, so

$$b_0 = 1. \quad (119)$$

For the second pair, $10 \leq 30$, so

$$b_1 = 0. \quad (120)$$

The recovered bit sequence is 10.

18.3 2-FSK phase-discriminator example

Suppose the per-symbol averaged discriminator levels are

$$v = \{-0.24, 0.27, -0.22, 0.25\} \text{ rad/sample.} \quad (121)$$

The median lies near

$$T \approx 0.015 \text{ rad/sample.} \quad (122)$$

Therefore the decisions are

$$\{0, 1, 0, 1\}. \quad (123)$$

If the sample rate were 2 MS/s, a discriminator value of 0.25 rad/sample would correspond to the frequency offset

$$f = \frac{2 \times 10^6}{2\pi} (0.25) \approx 79.6 \text{ kHz.} \quad (124)$$

The helper does not need to perform this Hz conversion to distinguish the two tones.

18.4 SSTV VIS timing and tone-score example

Suppose the SSTV VIS Samples/Symbol value is

$$N_s = 7500. \quad (125)$$

The derived input sample rate is

$$F_{\text{in}} = \frac{7500}{0.030} = 250000 \text{ samples/s.} \quad (126)$$

The audio decimation factor is

$$D = \left\lfloor \frac{250000}{24000} \right\rfloor = 10, \quad (127)$$

so the effective audio rate is $F_a = 25000$ samples/s. If one VIS data window produces

$$S(1100) = 0.74, \quad S(1300) = 0.13, \quad (128)$$

then the recovered decision for that interval is 1 because the 1100 Hz score is larger. A data interval with $S(1100) < S(1300)$ is decoded as 0.

18.5 QPSK nearest-point example

After normalization, suppose a received symbol is

$$r = 0.68 + 0.73j. \quad (129)$$

The normalized QPSK point in the first quadrant is

$$p_0 = \frac{1+j}{\sqrt{2}} \approx 0.707 + 0.707j. \quad (130)$$

Its distance is

$$d_0 = |r - p_0| \quad (131)$$

$$\approx \sqrt{(0.68 - 0.707)^2 + (0.73 - 0.707)^2} \quad (132)$$

$$\approx 0.035. \quad (133)$$

If that is the minimum distance among all four ideal points, the symbol receives the corresponding constellation label.

19 Engineering Interpretation and Assumptions

19.1 Fixed timing is an input assumption

The script assumes the supplied `sps` correctly identifies the symbol period and that the requested starting sample is acceptably aligned. Its averaging functions use nonoverlapping blocks beginning exactly at that offset. Therefore timing error changes which samples enter each symbol average.

19.2 SSTV VIS uses a 30 ms timing interpretation

For `sstv_vis`, `sps` is not treated as an arbitrary digital symbol length. It is interpreted as the number of IQ samples occurring during one 30 ms VIS bit. Consequently, an incorrect value changes the inferred input sample rate and therefore shifts every audio-tone correlation frequency. The top-level SSTV path examines up to five seconds of IQ beginning at `start-sample`; the complete 0.910 s header must lie inside the samples that are read.

19.3 No explicit carrier recovery in the constellation fallback

The PSK/QAM fallback performs amplitude normalization but does not estimate a static phase rotation or residual carrier-frequency rotation. If

$$r_k = s_k e^{j\theta}, \quad (134)$$

with significant unknown θ , normalization alone cannot remove θ . This helper consequently expects the input constellation to be sufficiently aligned for the chosen decision points.

19.4 Differential phase naturally suppresses amplitude scale

Because

$$\arg\{\alpha x[n](\alpha x[n-1])^*\} = \arg\{|\alpha|^2 x[n]x^*[n-1]\}, \quad (135)$$

any positive common gain factor disappears from the phase result. This is why the FSK discriminator fundamentally depends on phase progression rather than received amplitude.

19.5 Median FSK threshold adapts to common offset

If every FSK decision statistic receives a common offset c ,

$$v'_k = v_k + c, \quad (136)$$

then

$$\text{median}(v') = \text{median}(v) + c. \quad (137)$$

Therefore the binary comparisons are unchanged as long as the two tone clusters remain separable. This gives the implemented 2-FSK slicer tolerance to a common discriminator bias.

19.6 K-means estimates tone levels from the recording

The 4-FSK path does not require four predetermined phase-increment values. It estimates the four centers from the observed symbol statistics. This allows a common frequency offset and unknown deviation scale to move the cluster locations without requiring hard-coded frequencies, although the cluster populations still need to be distinguishable.

19.7 Constellation labeling is implementation-defined by point order

For the NumPy fallback, `argmin` returns the index of the nearest point in the arrays created by `constellation_points`. That index is subsequently converted directly to bits. The resulting label order is therefore the source array order; it is not a separate protocol-specific Gray mapping or differential decoder.

20 DSP Function Reference Summary

Function	Primary mathematics	Role
<code>read_complex16_raw</code>	$(I + jQ)/32768$	Load and normalize interleaved complex16 IQ
<code>symbol_views</code>	block means; mean $ x ^2$	Build complex and power symbol statistics
<code>normalize_complex</code>	$x/\sqrt{E x ^2}$	Unit-average-power normalization
<code>normalize_float</code>	$(x - x_{\min})/(x_{\max} - x_{\min})$	Scale real data to $[0, 1]$
<code>run_ook_symbol_slicer</code>	midpoint threshold	OOK/ASK symbol decisions
<code>threshold_binary_robust</code>	percentile midpoint	Robust RAW envelope state detection
<code>build_runs_from_states</code>	run-length encoding	Convert states to pulse durations
<code>merge_short_runs</code>	duration threshold	Suppress short threshold fragments
<code>decode_pwm_pairs_from_runs</code>	$L_H > L_L$	Decode relative-width RAW OOK pulses
<code>run_ook_raw_pulse_decoder</code>	envelope + smoothing + RLE	Complete RAW OOK decode chain
<code>phase_discriminator</code>	$\arg(x[n]x^*[n-1])$	Instantaneous phase increment
<code>sstv_fm_audio</code>	mean removal + block-average decimation	Produce normalized SSTV audio and effective rate
<code>sstv_tone_power</code>	$ \sum x[n]w[n]e^{-j\omega n} ^2$	Measure one candidate SSTV tone
<code>sstv_tone_score</code>	target power / candidate-tone power	Normalize 1100/1200/1300/1900 Hz evidence
<code>run_sstv_vis</code>	timed tone scores + parity check	Detect VIS header and recover seven data bits plus parity
<code>moving_average</code>	rectangular FIR convolution	Smooth envelope/discriminator data
<code>symbol_average_float</code>	block mean	Integrate one real statistic per symbol
<code>run_fsk2</code>	median slice of phase increments	2-FSK/GFSK/AFSK decisions
<code>kmeans_1d</code>	nearest-center assignment + centroid mean	Estimate discrete frequency levels
<code>run_fsk4</code>	4-center nearest decision	4-FSK symbol labels
<code>constellation_points</code>	ideal PSK/QAM geometry	Build reference constellations
<code>run_constellation_demod</code>	$\arg \min_j r - p_j $	PSK/QAM symbol decisions
<code>symbol_to_bits</code>	shifts, masks, XOR	Convert symbol labels to bits

21 Core DSP Source Listings

This section groups the principal DSP calculations in source order for quick reference. The line numbers are the actual line numbers in the supplied `gnuradio_decode_file.py`.

21.1 Complex16 normalization and symbol statistics

```

65     with path.open("rb") as fp:
66         fp.seek(start_sample * 4, 0)
67         raw = np.fromfile(fp, dtype=np.int16, count=count_iq * 2)
75     iq = raw.reshape((-1, 2)).astype(np.float32)
77     return ((iq[:, 0] + 1j * iq[:, 1]) / 32768.0).astype(np.complex64)
92     c = read_complex16_raw(path, start_sample, max_symbols * sps)
93     usable = (c.size // sps) * sps
102    c = c[:usable]
103    blocks_view = c.reshape((-1, sps))
104    complex_symbols = blocks_view.mean(axis=1).astype(np.complex64)
105    mag_symbols = np.mean(np.abs(blocks_view) ** 2, axis=1).astype(np.float32)

```

21.2 Normalization

```

117    p = float(np.mean(np.abs(x) ** 2))
118    if p <= 0.0 or not math.isfinite(p):
119        return x.astype(np.complex64)
120    return (x / math.sqrt(p)).astype(np.complex64)
130    lo = float(np.min(x))
131    hi = float(np.max(x))
132    if not math.isfinite(lo) or not math.isfinite(hi) or hi <= lo:
133        return x.astype(np.float32)
134    return ((x - lo) / (hi - lo)).astype(np.float32)

```

21.3 RAW OOK DSP

```

228    for low_pct, high_pct in ((5, 95), (10, 90), (15, 85), (20, 80), (25, 75)):
229        lo = float(np.percentile(vals, low_pct))
230        hi = float(np.percentile(vals, high_pct))
232        threshold = lo + 0.5 * (hi - lo)
233        states = (vals >= threshold).astype(np.uint8)
263        if state_a == 1 and state_b == 0:
265            if len_a >= min_len and len_b >= min_len:
266                bits.append(1 if len_a > len_b else 0)
298    env = (np.abs(samples) ** 2).astype(np.float32)
299    smooth_len = max(1, min(max(1, sps // 20), 128))
300    env_smooth = moving_average(env, smooth_len)
304    states = threshold_binary_robust(env_smooth)
305    runs = build_runs_from_states(states)
306    runs = merge_short_runs(runs, max(1, sps // 10))
307    bits = decode_pwm_pairs_from_runs(runs, max(1, sps // 5))

```

21.4 SSTV VIS DSP

```

345    discriminator = phase_discriminator(samples).astype(np.float64)
349    discriminator -= np.mean(discriminator)
352    decimation = max(1, int(math.floor(input_rate / target_rate)))
357    audio = discriminator[:usable].reshape((-1, decimation)).mean(axis=1)
377    x *= np.hanning(x.size)
380    angle = 2.0 * math.pi * frequency * n / sample_rate
381    real = float(np.dot(x, np.cos(angle)))
382    imag = float(np.dot(x, np.sin(angle)))
384    return (real * real) + (imag * imag)

```

```

404 frequencies = (1100.0, 1200.0, 1300.0, 1900.0)
409 return powers[target_index] / total
421 input_rate = float(sps) / 0.030
439 sstv_tone_score(audio, audio_rate, t + 0.050, 0.200, 1900.0)
440 + sstv_tone_score(audio, audio_rate, t + 0.300, 0.010, 1200.0)
457 one_score = sstv_tone_score(audio, audio_rate, t, 0.020, 1100.0)
458 zero_score = sstv_tone_score(audio, audio_rate, t, 0.020, 1300.0)
469 if ((sum(bits) + parity) & 1) != 0:
471 if stop_score < 0.45:
474 bits.append(parity)

```

21.5 FSK-family DSP

```

336 x = normalize_complex(samples)
337 return np.angle(x[1:] * np.conj(x[:-1])).astype(np.float32)
486 kernel = np.ones(n, dtype=np.float32) / float(n)
487 return np.convolve(x, kernel, mode="same").astype(np.float32)
500 vals = x[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
511 freq = phase_discriminator(samples)
512 if gfsk:
513     freq = moving_average(freq, max(3, sps // 3))
514 vals = symbol_average_float(freq, sps, max_symbols)
517 threshold = float(np.median(vals))
518 return [1 if v >= threshold else 0 for v in vals]

```

21.6 4-FSK clustering

```

533 centers = np.percentile(vals, np.linspace(5, 95, k)).astype(np.float32)
534 for _ in range(rounds):
535     dist = np.abs(vals[:, None] - centers[None, :])
536     labels = np.argmin(dist, axis=1)
539     part = vals[labels == idx]
540     if part.size:
541         new_centers[idx] = float(np.mean(part))
545 return np.sort(centers)
555 vals = symbol_average_float(phase_discriminator(samples), sps, max_symbols)
558 centers = kmeans_1d(vals, 4)
559 labels = np.argmin(np.abs(vals[:, None] - centers[None, :]), axis=1)
560 return [int(v) & 0x3 for v in labels]

```

21.7 PSK/QAM decisions

```

568 if mod == "bpsk":
569     return np.array([-1 + 0j, 1 + 0j], dtype=np.complex64), 1
570 if mod == "qpsk":
571     return np.array([1 + 1j, -1 + 1j, -1 - 1j, 1 - 1j], dtype=np.complex64), 2
572 if mod == "psk8":
573     pts = [np.exp(1j * 2.0 * math.pi * n / 8.0) for n in range(8)]
575 if mod == "qam16":
576     levels = [-3, -1, 1, 3]
577     pts = [complex(i, q) for q in levels for i in levels]
579 if mod == "qam64":
580     levels = [-7, -5, -3, -1, 1, 3, 5, 7]
581     pts = [complex(i, q) for q in levels for i in levels]
596 x = normalize_complex(samples) if normalize else symbols.astype(np.complex64)
620 pts, bps = constellation_points(mod)
621 pts = normalize_complex(pts)
622 labels = np.argmin(np.abs(x[:, None] - pts[None, :]), axis=1)

```